

Forecast sorting & reservation queue logic

Forecast page sorting — the exact algorithm

`_get_report_data` builds the report in five sequential stages. Order matters — each stage consumes what's left after the previous one pulled its group out.

Stage 0 — merge fragments. Before anything else, unreserved rows sharing the same `move_out` id get merged into one (summing `quantity`, OR-ing `clearance_reserve_is_misleading`). Native's own per-move ins/outs reconciliation can otherwise fragment a single line's remaining demand into multiple rows.

Stage 1 — split reserved vs. unreserved. `line.get("reservation")` truthy → reserved block. Everything else → unreserved. This split is absolute: an unreserved line *never* outranks a reserved one, regardless of clearance date. (Real bug this fixed: a far-future order excluded from actual reservation could still have an *early* `clearance_date` and would sort above a genuinely-holding order under pure clearance sorting.)

Stage 2 — reserved block sorts by `by_delivery_date`. Key = `(0, move_out.date)` if a delivery date exists, else `(1, "")` — Python's sort is stable so no-date rows keep encounter order. *Not* clearance date — once you're holding stock, priority already did its job; only "when do you need it" remains interesting.

Stage 2b — extract split-remainder siblings. Any unreserved row whose `move_out.id` matches a reserved row's `move_out.id` is pulled out of the unreserved pool into `split_siblings_by_move_id`, keyed by that move id. This is *before* any unreserved sorting runs, so a split line's leftover portion never gets scattered by clearance/delivery sort — it's reunited later purely by `move_out` id, not by any sort key.

Stage 3 — no-clearance force-reserved floats to the top of unreserved. `lock_reason == "Force Reserved"` and `line_clearance_date(line)` is None — a `no_invoice` order that's *only* force-reserved (no real payment at all) still holds an active claim on whatever arrives next. Sorted among itself by `lock_date` (when the force-reserve was applied), ascending.

Stage 4 — extract far-out. From what's left, `lock_reason == "Scheduled Far Out"` rows are pulled out entirely — these had a genuine `clearance_date` at some point (the defer reason structurally can't apply to a `no_invoice` order — `_compute_clearance_defer_reason` already zeroes it out for those before the far-out check even runs) but are excluded from competing right now.

Stage 5 — cleared vs. uncleared, on what remains.

- `cleared_unreserved` = `line_clearance_date(line)` is not None → sorted by `clearance_sort_key` : `(0, clearance_date)` if present else `(99, datetime.max)`.
- `uncleared_unreserved` = no clearance date at all → sorted by `by_delivery_date` (pure informational tiebreak among themselves — there's no real queue position to sort by).

Final assembly, reserved-remainder siblings re-spliced in immediately after their own reserved row:

```
reserved_block (each reserved row + its own split siblings directly after it)
+ no_clearance_force_reserved
+ cleared_unreserved_sorted
+ far_out_unreserved_sorted
+ uncleared_unreserved_sorted
```

So the five visible bands, top to bottom: **holding stock** → **force-reserved with no ticket** → **paid, waiting, by ticket order** → **paid but excluded (far out)** → **unpaid**.

Reservation queue — the exact algorithm

`_reserve_by_clearance(product_ids=None)`, run inside `self.with_context(_within_reserve_by_clearance=True)` to prevent re-entrant recursion from its own side-effect writes:

1. Eligible orders domain:

```
fulfillment_stage in (order_pick, ship, grace_period) OR order_line.is_force_reserved = True
```

`no_invoice` is never included — hard lock grants no exception to this; it's checked separately for a completely different purpose (protecting, not acquiring).

2. `all_moves` — same stage/force-reserve condition, at the move level, `state` not in (`done`, `cancel`) , `sale_line_id` set, scoped to `product_ids` if given.

3. `foreign_moves` — moves currently `assigned` / `partially_available` whose order is `no_invoice` , not hard-locked (order or line), not force-reserved. These exist only via a path this module doesn't gate (native's forecast Reserve link ignores payment status) or pre-existing data — included solely to be reclaimed, never to join orders .

4. `far_future_moves` — `_clearance_is_far_future_move` : `pick_scheduled_date > now + 6 months` , and not hard-locked/force-reserved (an override always sticks regardless of scheduling).

5. Fast path — if every move in `all_moves` is already `assigned` , no `foreign_moves` exist, and no far-future move is sitting `assigned` , return immediately. No release/reassign needed.

6. `pre_release_free_qty` snapshot — for every product with at least one `Scheduled Future Stock` holder in the touched set, `product.with_context(location=warehouse.lot_stock_id).free_qty` , taken before step 7's release. This is the signal that later tells a genuine new arrival apart from this run's own churn.

7. Blanket release. `protected = is_locked_reservation` moves, hard-locked order/line moves, `clearance_defer_reason == "Scheduled Future Stock"` moves. Everything else in (`touched = all_moves | foreign_moves`) that's `assigned` / `partially_available` gets `_do_unreserve()` 'd — unconditionally, every run.

8. Exclude far-future from `all_moves` (they never get a reassignment attempt at all).

9. Main allocation pass. Group remaining `all_moves` by product; within each product, sort by `_clearance_move_priority` — (0, `pick_scheduled_date`) for `Scheduled-Future-Stock` holders, (1, `force_reserved_date`) , (2, `clearance_date`) , (3, `datetime.min`) — call `_action_assign()` in that exact order, one move at a time, so each move only ever gets what's left after every higher-priority move already took its share. Any move that ends up `assigned` / `partially_available` and qualifies for a lock (hard-locked or force-reserved) gets `is_locked_reservation=True` right here, even if it was already reserved before this run.

10. Targeted release pass (per product, only where `Scheduled Future Stock` holders exist). For each still-unfulfilled demand move (not force-reserved — force-reserve can never trigger this path), sorted by priority:

- Skip if `pick_scheduled_date` is unset.
- `eligible_holders` = current holders where: `state` is `assigned` / `partially_available` , not (`is_scheduled_future_stock_release` and `pre_release_free_qty > 0` for that product) — i.e. exclude a holder that just legitimately reclaimed real new stock this same run — and holder's own `pick_scheduled_date > demand.pick_scheduled_date` .
- Sorted by holder's `pick_scheduled_date` descending (most slack first) — release one at a time, only as much as needed, flagging `is_scheduled_future_stock_release = True` on each released holder.
- After the demand loop, every holder gets a chance to reclaim: `_action_assign()` in priority order (tier 0 first) — clears the flag if fully reclaimed; clears it anyway (falls back to ordinary priority) if the backing PO evaporated (`not _has_safe_future_replacement()`).

11. Never touches `fulfillment_stage` (Ship promotion/demotion is payment-driven only, handled in `account_move.py`). Posts one consolidated chatter message per picking summarizing what was reserved/released and why (tier label).

`_get_group_safe_future_replacement_lines` (what decides who's even eligible to be a `Scheduled-Future-Stock` candidate, called from `_compute_clearance_defer_reason`): groups every candidate line by (`product`, `warehouse`) , sorts by `pick_scheduled_date` ascending, and for each line in turn checks whether the shared committed-incoming pool — after every earlier (sooner-scheduled) candidate in the same group already deducted its

own claim — still covers this line's full quantity by its own matched shipment date, with the confirmed/unconfirmed buffer applied. This is what closes the "two lines both individually pass the check against the same shipment" overcommit bug — it's a group decision, computed once per batch, not per line in isolation.